So you want to use the vertex model?

Guide to how the AGVertex Model works

Basic Story

- CPP Code Base contains the model that I (Avik) inherited at the beginning of my Ph.D (TissueClass, EdgeClass, VertexClass, VectClass)

- Everything that I have done since “inherits” code from those base classes via inheritance and polymorphism, as implemented in C++

- Prerequisites to using the model:
  - a basic understanding of MATLAB,
  - Python (if you want to build out the small amount of Python I have implemented),
  - a decent understanding of Object Oriented Programming (pointers, objects, functions, inheritance, polymorphism, string manipulation,)

Generation	Vertex Model Classes					Projects that use them
I	TissueClass	EdgeClass	CellClass	VertexClass	VectClass	CPP Code Base, Tutorial
II	AGTissue		AGCell	-	-	AGVertexModel,
III	AGPerimTissue	AGPerimEdge	AGPerimCell	-	-	AGPerimVertex, AGPerimPolydispersityCalc, AGPerimHessian_test, AGPerimAffine_test
IV	AGStiffTissue	AGStiffEdge	AGStiffCell	-	-	AGStiffness, AGStiffEquilibrate, AGProliferate

Generation	Vertex Model Classes					Projects that use them
I		eClass	CellClass	VertexClass	VectClass	CPP Code Base, Tutorial
II	AGTissue		AGCell			AGVertexModel,
III	AGPerimTissue	AGPerimEdge	AGPerimCell			AGPerimVertex, AGPerimPolydispersityCalc, AGPerimHessian_test, AGPerimAffine_test
IV	AGStiffTissue	AGStiffEdge	AGStiffCell			AGStiffness, AGStiffEquilibrate, AGProliferate

I use “Generation” to distinguish between the different sets of projects I worked on in graduate school. (More on this in later slides).

Alternatively, you can roughly think of each Generation as a different version of the vertex model.

The green arrows denote “inheritance.” i.e., AGTissue is a child class or subclass of TissueClass.

All subclasses can use the features of their parent or grandparent or greatgrandparent classes, unless those features were explicitly replaced.

Generation	Projects that use them	Features	Purpose
I	CPP Code Base, Tutorial	- Constant tension - Euler, RK4, RK4-Adaptive integrators - (isotropic) Mitosis, Apoptosis, T1's	Inherited from Meryl, useful first case for starting out with Visual Studio Code
II	AGVertexModel,	- Grows tissue via oriented divisions - Set external stress along x or y-dimension - Allows box walls to adjust independently - Calculates internal stress tensor	Allows for anisotropic, noisy growth with considerations of external and internal stress
III	AGPerimVertex, AGPerimPolydispersityCalc, AGPerimHessian_test, AGPerimAffine_test, AGPerimModuli	- Tension is linear in perimeter (traditional vertex model energy) - Integrator can backtrack to avoid overshooting local minima and more precisely reach equilibrium - Shear modulus calculation (via squeeze is implemented, via analytic equation not finished) - Pressure can be made quartic, logarithmic, exponential, or quadratic	Displays the jamming transition when preferred perimeter is made small. We used it to probe compression induced fluidization. This model can be made to grow. Cells could have heterogeneous moduli or preferred areas/perimeters (HESSIAN CODE MIGHT NOT ASSUME THIS THOUGH)
IV	AGStiffness, AGStiffEquilibrate, AGProliferate	- Cells have heterogeneous properties (K, Γ, A_0) based on location in box	Used to model fly wing cell pattern development

Generation	Projects that use them	Features	Purpose
I	CPP Code Base, Tutorial	- Constant tension - Euler, RK4, RK4-Adaptive integrato - (isotropic) Mitosis, Apoptosis, T1's	Inherited from Meryl, useful first case for starting out with Visual Studio Code
II	AGVertexModel,	- Grows tissue via oriented divisions - Set external stress along x or y-dimension - Allows box walls to adjust independently - Calculates internal stress tensor	Allows for anisotropic, noisy growth with considerations of external and internal stress
III	AGPerimVertex, AGPerimPolydispersityCalc, AGPerimHessian_test, AGPerimAffine_test, AGPerimModuli	- Tension is linear in perimeter (traditional vertex model energy) - Integrator can backtrack to avoid overshooting local minima and more precisely reach equilibrium - Shear modulus calculation (via squeeze is implemented, via analytic equation not yet correct) - Pressure can be made quartic, logarithmic, exponential, or quadratic	Displays the jamming transition when preferred perimeter is made small. We used it to probe compression induced fluidization. This model can be made to grow. Cells could have heterogeneous moduli or preferred areas/perimeters (HESSIAN/ANALYTIC SHEAR CODE MIGHT NOT ACCOUNT FOR THIS THIS THOUGH)
IV	AGStiffness, AGStiffEquilibrate, AGProliferate	- Cells have heterogeneous properties (K, Γ, A_0) based on location in box (heterogeneity is defined by defining a field in the Lagrangian frame. Theoretically, Generation III should be able to implement heterogeneity as defined in the Eulerian frame).	Used to model fly wing cell pattern development

Here, we denote the purpose and corresponding features of every Generation.

Projects	Purpose
CPP Code Base	- Inherited from Meryl. Runs an outdated stress calculation and box growth set up.
Tutorial	- See Tutorial folders. (No cheating, do the tutorial :D)
AGVertexModel	- Equilibrates a cell packing. Allows for anisotropic growth via oriented divisions. Box walls can adjust independently. External pressure can be set independently. DOES NOT CALCULATE OR DEFINE AN EQUILIBRIUM
AGPerimVertex	- Equilibrates a cell packing. Can allow for anisotropic growth and box walls adjusting. Uses the more traditional vertex model energy you find in most papers. Has a integrator backtracker that prevents the model from overshooting local minima. This was necessary for correctly detecting the jamming transition in the vertex model
AGPerimModuli	- This code is used to shear a packing and re-equilibrate it. The point is to calculate the energy difference before and after the shear and to use that to calculate the vertex model
AGPerimHessian_test	- This is used to develop the analytical shear modulus calculation. It doesn't work (but it seems darn close).
AGPerimAffine_test	- This affinely deformed an equilibrated packing and calculated the change in energy (used to validate affine part of shear modulus code)
AGPerimPolydispersity Calc	- Used to calculate the polydispersity of tissues
AGStiffness	- Sets a chosen fixed-in (Lagrangian) pattern of either K, A0, Gamma and runs an equilibrated packing. The packing should be equilibrated while homogeneous, then you stick it into this project when you want to make it heterogeneous
AGStiffEquilibrate	- Equilibrates the tissue before getting fed to the AGStiffness project. Might not be necessary, but that's what it's set up for.
AGProliferate	- Takes a homogeneous tissue and grows it to a chosen size OR number of cells. We never made it very far with this, I have no clue if it works.

Projects	Purpose
CPP Code Base	- Inherited from Meryl. Runs an outdated stress calculation and box growth set
Tutorial	- See Tutorial folders. (No cheating, do the tutorial :D)
AGVertexModel	- Equilibrates a cell packing. Allows for anisotropic growth via oriented divisions can be set independently. DOES NOT CALCULATE OR DEFINE AN EQUILIBRIUM
AGPerimVertex	- Equilibrates a cell packing. Can allow for anisotropic growth and box walls adju find in most papers. Has a integrator backtracker that prevents the model fror correctly detecting the jamming transition in the vertex model
AGPerimModuli	- This code is used to shear a packing and re-equilibrate it. The point is to calcul to use that to calculate the vertex model
AGPerimHessian_test	- This is used to develop the analytical shear modulus calculation. It doesn't wo
AGPerimAffine_test	- This affinely deformed an equilibrated packing and calculated the change in e
AGPerimPolydispersity Calc	- Used to calculate the polydispersity of tissues
AGStiffness	- Sets a chosen fixed-in (Lagrangian) pattern of either K, A0, Gamma and runs an equilibrated packing. The packing should be equilibrated while homogeneous, then you stick it into this project when you want to make it heterogeneous
AGStiffEquilibrate	- Equilibrates the tissue before getting fed to the AGStiffness project. Might not be necessary, but that's what it's set up for.
AGProliferate	- Takes a homogeneous tissue and grows it to a chosen size OR number of cells. We never made it very far with this, I have no clue if it works.

C++ requires a main function to run any code. These main functions use the vertex model classes to carry out a specific task.

I define each of these main files and the corresponding vertex model classes as a "Project."

To be extra clear, these projects don't correspond to versions of the vertex model.

So you want to use the vertex model?

- The above slides give an overview for using the C++ code
- If you want to use the vertex model, you should follow a few principles of responsible code reuse and design.
- This topic fills out entire textbooks, so here I'll give you some very small tips
  - 1. Making projects that reuse existing code
  - 2. Making projects that require changes the vertex model

Reusing Old Code

- If you want to carry out a task that uses only code the vertex model already has, you need to make a new project
- It's best to not mess with my existing code (in case it needs to get reused)
- The one exception is if you're trying to fix the analytical shear modulus calculation in Generation III.

Projects	Purpose
CPP Code Base	- Inherited from Meryl. Runs an outdated stress calculation and box growth set up.
. . .	. . .
AGStiffEquilibrate	- Equilibrates the tissue before getting fed to the AGStiffness project. Might not be necessary, but that's what it's set up for.
AGProliferate	- Takes a homogeneous tissue and grows it to a chosen size OR number of cells. We never made it very far with this, I have no clue if it works.
Your Project	- Your target task

Generation	Projects that use them	Features	Purpose
I	CPP Code Base, Tutorial	- Constant tension - Eule - (iso	Inherited from Meryl, useful first case udio Code
II	AGVertexModel, Your Project?	- Gro - Set - Allo - Calc	y growth rnal and
III	AGPerimVertex, AGPerimPolydispersityCalc, AGPerimHessian_test, AGPerimAffine_test, AGPerimModuli	- Tens ene - Inte min - She via - Pres or q	on when small. We induced e made to geneous rimeters ASSUME
IV	AGStiffness, AGStiffEquilibrate, AGProliferate, Your Project?	- Cells have heterogeneous properties (K, Γ, A_0) based on location in box	Used to model fly wing cell pattern development

You'll have to put some thought in to which Generation your new project fits into.

For the most part, all the code in older generations can be used by the newer generations.

The biggest exception to this rule is the change in tension implementation between Generation II and Generation III.

If you want a constant tension model, you should implement your new Project in Generation II. This means you only use the classes in each generation.

If you can want to use the perimeter squared tension, you can incorporate your project in Generation IV.

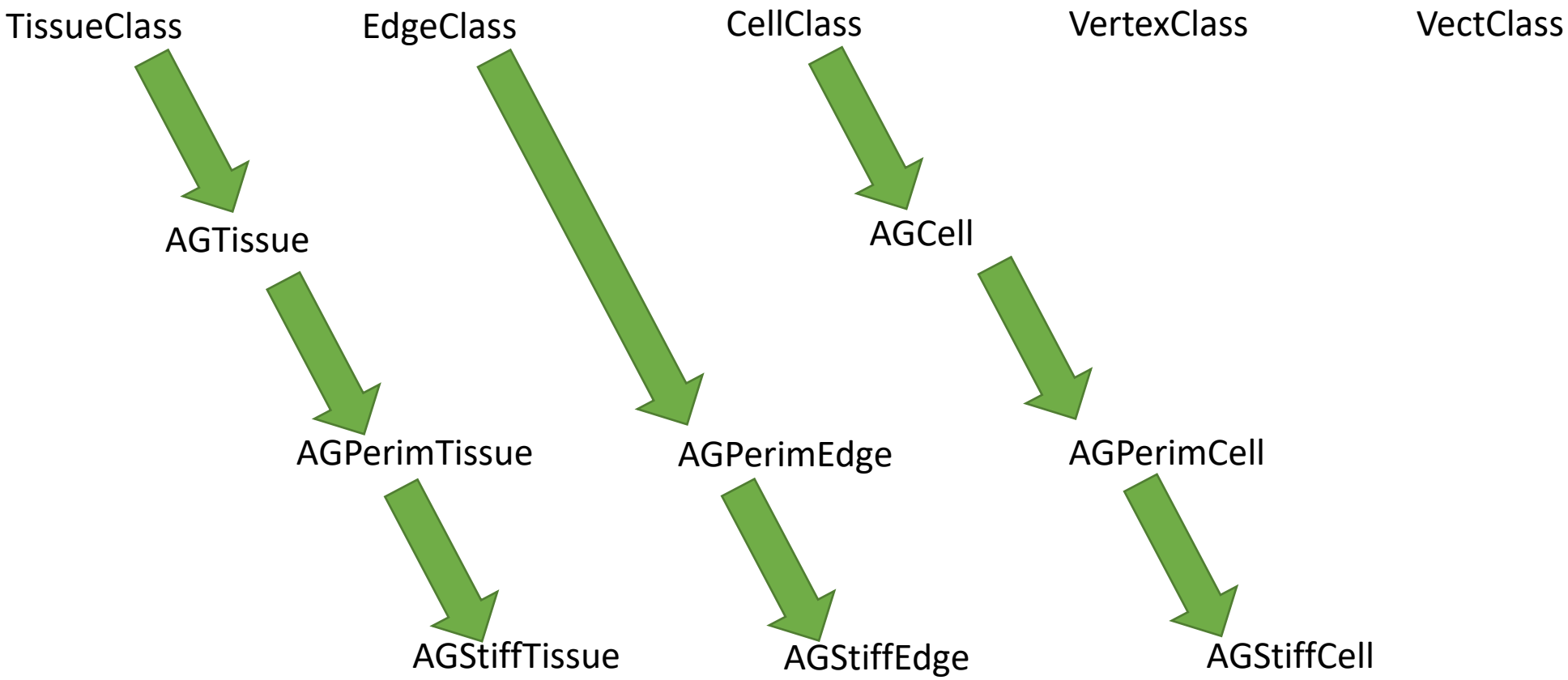
Making New Code

- If you need to implement a new feature, you need to make a new project AND you need to make some new classes
- The main question becomes “where” you introduce this new class
- The vertex model Tutorial shows you how to do this. If you have experience with C++, the Tutorial is still important to go through because of the unique features of the VertexModel. For example, every time you make a new type of edge, you need to update (rewrite) classes that create new edges, and the T1 code. You can determine exactly which functions need to be rewritten by studying the classes that are REWRITTEN between AGPerimTissue and AGTissue

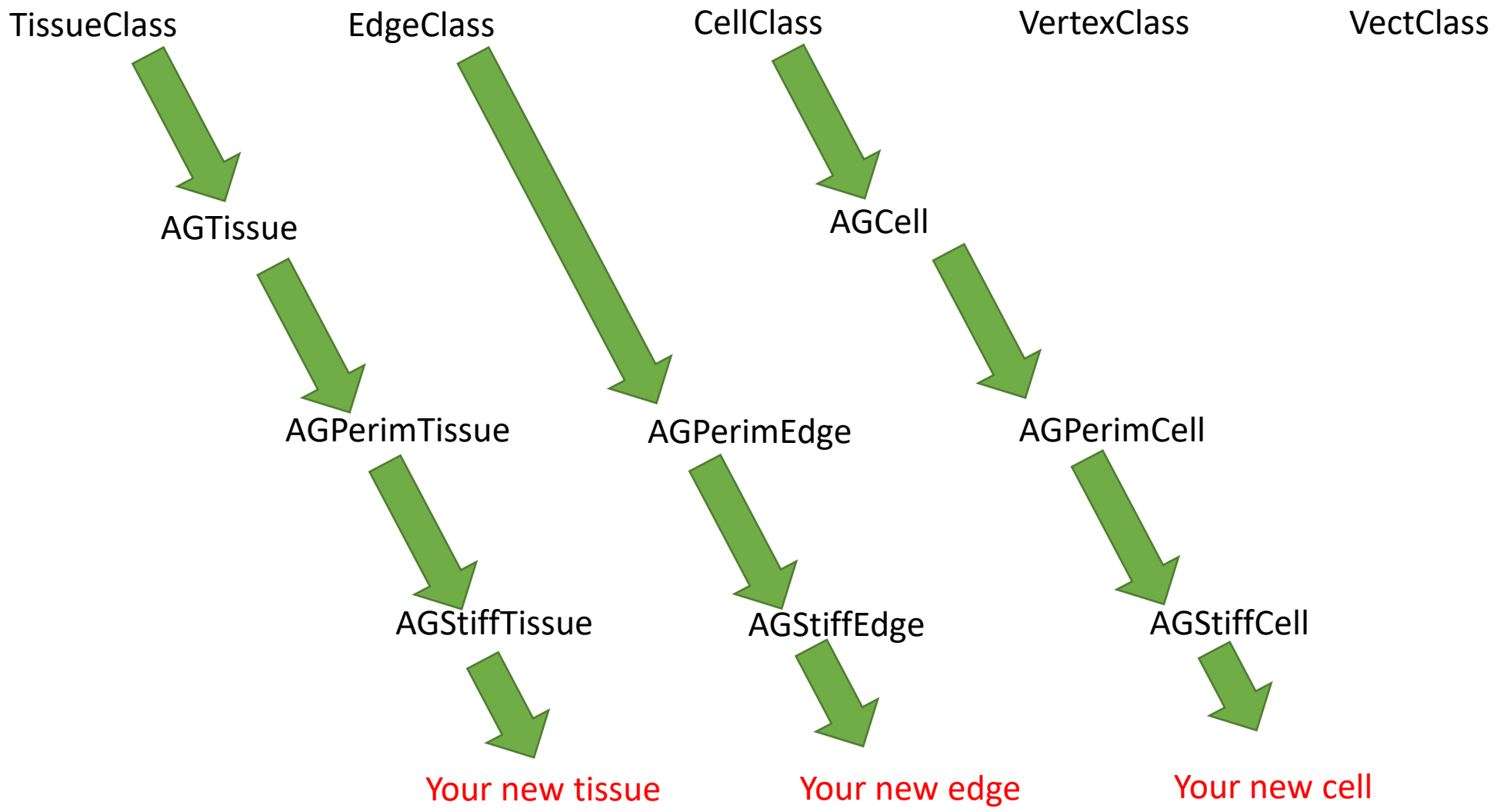
Generation Structure of Vertex Model

- Currently, we can organize the vertex model very simply into generations because it is useful to stack functionality in subsequent versions of the code (i.e. there was no harm in making the AGPerimVertex model capable of growth, despite the fact that we did not explicitly need it)

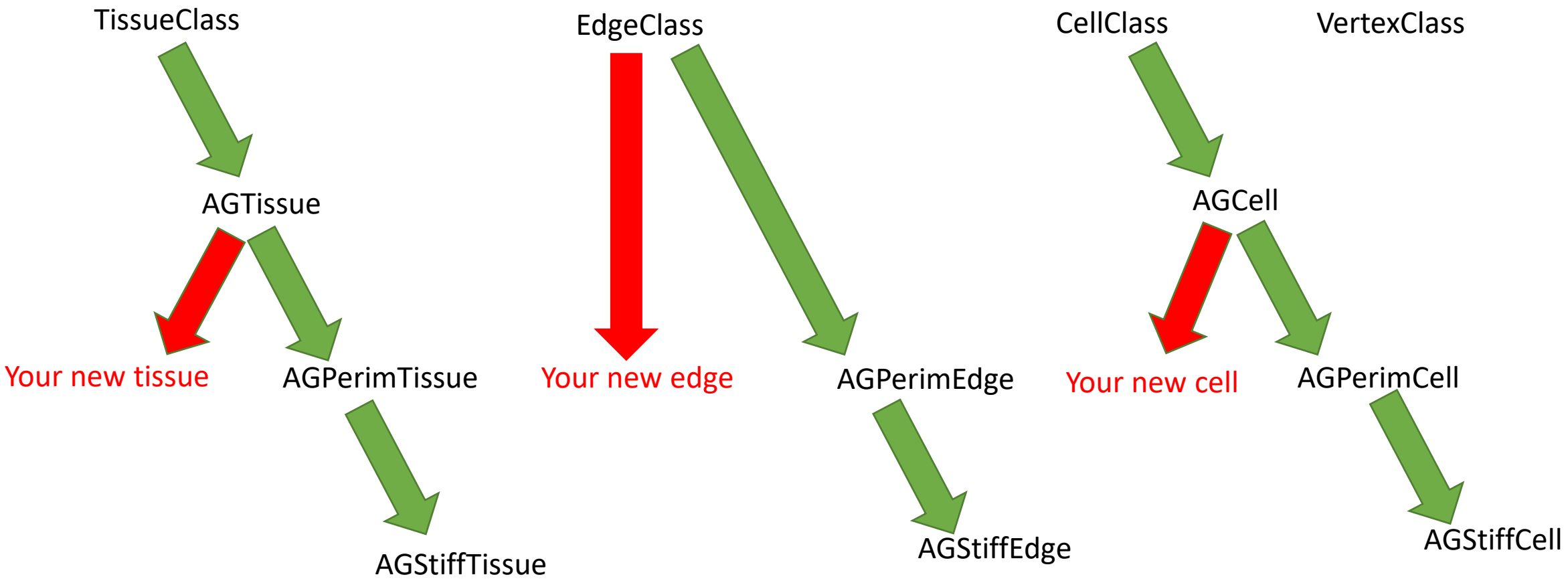
- However, there is a situation in which you might want the functions of older generations, but some your features might not complement the newer generations



Here, we again show the Inheritance Structure of the existing vertex code. Again, each row corresponds to a Generation of my vertex model collection.



If you want to take all the features of the previous Generations, you should just add new classes that depend on AGStiffTissue, AGStiffEdge, AGStiffCell.



Since Generation II has constant tension and later Generations have the quadratic perimeter, you may want to make a new “branch” to the model. The “new” classes have the features of Generation II (constant tension) but also none of the features of Generation III and IV. The new classes are subclasses of the classes of Generation II.

You would then distinguish your new classes from the AGPerim classes (may call the AGPerimClasses Gen III, branch AGPerim versus Gen III, branch *insert your name here*

Parameter	Definition/Options	Generation
InputFileName	File name for cell tessellation, developed by MATLAB files	1,2,3,4
INTEGRATORNAME	Defines what kind of numerical integration the model implements. Options: “Euiler” (misspelling for Euler, the default), “RK4” (Runga Kutta Fourth Order), “RK4-Adaptive” (Varies RK4 timestep to increase efficiency of run time)	1,2,3,4
LX	Box dimension in x-direction	1,2,3,4
LY	Box dimension in y-direction	1,2,3,4
MINAREA	Smallest allowed area for a cell. If a cell wants to get smaller than this, it will undergo apoptosis	1,2,3,4
A0	Preferred area of cells. For Gen IV, this gives the average preferred area if you choose to vary preferred area	1,2,3,4
MINLENGTH	Smallest allowed edge length for a cell. If the edge wants to get smaller than this, it will go through a T1. (DEFAULT = 1)	1,2,3,4
NEWEDGELENGTH	Edge length for when T1 makes a new edge. (DEFAULT = 5). If you don’t make the new edge bigger than MINLENGTH, there is a possibility of getting oscillating T1’s. (See Meryl’s thesis)	1,2,3,4
SEED	No idea. Possibly a relic of a version of the code that Meryl used but didn’t pass down to me.	1
K (pressure units)	The scaling factor for the energetic cost of changing the area of a cell. Multiply this by (1 – A/A0) to get the cell pressure	1,2,3,4
GAMMA (force units)	The strength of the constant tension. Multiply by length to get the edge contribution of the vertex model energy (in Gen 1 and Gen 2)	1,2,3,4
INITIALSTEPSIZE	The initial integration time step for the vertex model	1,2,3,4
TOTALTIME	Total time the model runs for	1,2,3,4
CURRENTTIME	Records the current total timestep of integration while the model runs	1,2,3,4

Parameter	Definition/Options	Generation
TIMEINTERVAL	Sets the time interval between output recording (The state of the model is written to the “out” file once per time interval)	1,2,3,4
TISSUE_ALPHA	Sets the mobility factor for the change in box dimensions. Gives the rate at which forces on the box walls make it grow	1,2,3,4
alphaM	Sets the mobility factor for the movement of the vertices. Gives the rate at which forces on the vertices move them	1,2,3,4
TISSUE_EXT_STRESS	Isotropic stress on box walls	1
TISSUE_EXT_STRESS_X	Stress on the vertical box walls	1,2,3,4
TISSUE_EXT_STRESS_Y	Stress on the horizontal box wall	1,2,3,4
TISSUE_EXT_STRESS_X_MAX	Max stress on vertical box walls. From before my time, I’ve never used this.	1
TISSUE_EXT_STRESS_Y_MAX	Max stress on horizontal box walls. From before my time, I’ve never used this.	1
error_tolerance_absolute_mechanical	No idea. It’s a function parameter for the RK4-Adaptive integration algorithm, but it’s unused. Maybe Hayden knows	1
LXnew	Holding variable for updated value of x box dimension when simulation is evolved.	1,2,3,4
LYnew	Holding variable for updated value of y box dimension when simulation is evolved.	1,2,3,4
TAU	No idea. Probably a variable Meryl forgot about, or used in a different project I don’t have access too.	1
TAU_STRESS	A timescale for when mitosis is allowed. I’ve never really used it because we switched to implementing mitosis whenever the model equilibrates, but this could be useful.	1,2,3,4
A0SKEW	Unused in CPP Cell Code Base.	1

Parameter	Definition/Options	Generation
T1PERCENT	Unused in CPP Cell Code Base.	1
MITOSISANGLE	Sets the orientation of the mitotic plane for mitosis. Not used, since AGCell code has a different parameter that it uses to set this property.	1
DIVISIONPOTENTIAL	Tells each cell how to pick mitosis angle. OPTIONS: “ani” (ANISOTROPIC POTENTIAL), “tcj” (TRICELLULAR JUNCTION), “combined” (the other two put together)	2, 3, 4
THETAAN	Sets the orientation of the most probably mitotic plane for mitosis if anisotropic OR combined mitosis is chosen	2,3,4
ANSTRENGTH	If the anisotropic potential is chosen, sets the variance of the distribution of division angles the model samples from. If combined potential is chosen, this defines the model’s tendency to set division angles as THETAAN	2,3,4
TCJSTRENGTH	If the tcj potential is chosen, sets the variance of the distribution of the division angles the model samples from. If tcj potential is chosen, this defines the model’s tendency to set division angles to that defined by the TCJ angle, determined by the geometry of the cells (See Bosveld et al 2016).	2,3,4
MITOSISON	OPTIONS: 0 (off), 1 (on). Tells model whether it should implement mitosis.	2,3,4
APOPTOTISON	OPTIONS: 0 (off), 1 (on). Tells model whether it should implement apoptosis.	2,3,4
WRITEMITOSIS	OPTIONS: 0 (off), 1 (on). Tells model whether it should record tissue state to “out” file after every mitosis	2,3,4
VERBOSEON	OPTIONS: 0 (off), 1 (on). Tells model whether the tissue state recording should be “verbose” or “succinct”(as defined by the code. See HelperWriteOutputData_Write2File(ofstream* datafile, bool includeExtraInfo) as defined by gen 2, 3, or 4	2,3,4

Parameter	Definition/Options	Generation
EQUILIBRIUMMITOSIS	OPTIONS: 0 (off), 1 (on). Tells the code whether to implement mitosis when the tissue is equilibrated.	2,3,4
pressureType	OPTIONS: “LOG”, “EXPONENTIAL”, “QUARTIC”, “QUARTIC2.0”, or any input. Sets the form of the pressure used in the model. Options refer to Logarithmic, exponential, quartic, quartic2.0, or quadratic (any put). Code is designed to default to quadratic if you feed in no input or unrecognized input. QUARTIC 2.0 defines the pressure potential with different scaling	2 (never been used in gen 2 though), 3, 4
C4	If one of the quartic potential is chosen, this sets the magnitude of the quartic term in the pressure stress energy	2,3,4
Xflag	OPTIONS: TRUE or FALSE. True if cell or edge crosses boundary in x-direction. Edges and Cells both have.	1, 2, 3, 4
Yflag	OPTIONS: TRUE OR FALSE. True if cell or edge cross boundary in the y-direction. Edges and Cells both have.	1, 2, 3, 4
currentEnergy	Energy at current running time for cells.	2,3,4
pastEnergy	Energy at previous timestep for cells.	2,3,4
P0	Preferred perimeter of cells	3,4
energyListSize	The amount of energy measurements the model stores for studying equilibria.	3,4
equilibriumtolerance	In theory, equilibrium happens when the the rate of change of energy in time goes to 0. In reality, it’s never 0. equilibriumtolerance sets the upper bound on the acceptable rate of change of energy in time	3, 4
LAMBDA	Sets the energetic cost of a change in perimeter squared. See vertex model hessian.	3, 4

Parameter	Definition/Options	Generation
Kstdratio	Sets spread of K for fixed in K heterogeneity	4
A0stdratio	Sets spread of A0 for fixed in A0 heterogeneity	4
PMstd	Sets spread of lambda for fixed in lambda heterogeneity	4
Ac	Another area scale that I implemented in Generation 4. We eventually decided that we didn't want to do things with this scale.	4